

Developers

Overview

Part-1

Ftrace

This article explains how to set up Ftrace and be able to understand how to trace functions. It should be useful for current kernel developers and device driver developers who want to debug kernel issues, and also for students who are keen to pursue a Linux systems programming career. The author assumes that readers possess good knowledge of the Linux kernel.

Ftrace (Function Tracer) is the 'Swiss army knife' of kernel tracing. It is a tracing mechanism built right into the Linux kernel. It has the capability to see exactly what is happening in the kernel, and debug it. Ftrace is more than a mere function tracer, and has a wide variety of tracing abilities to debug and analyse a number of issues like latency, unexpected code paths, performance issues, etc. It can also be used as a good learning tool.

Ftrace was introduced in kernel 2.6.27 by Steven Rostedy and Ingo Molnar. It comes with its own ring buffer for storing trace data, and uses the GCC profiling mechanism.

Prerequisites

You need a 32-bit or 64-bit Linux machine with a kernel development environment, and as new a kernel as possible (the newer the kernel, the more the tracing options you get). I use a Fedora Core 13 (x86_64) machine in my environment, but any distribution would suffice.

Setting up Ftrace

DebugFS needs to be set up to run on the machine

you want to use Ftrace on. If you are unaware of how to set up DebugFS, do refer to my DebugFS article in the October 2010 issue of *LINUX For You*.

DebugFS should have been mounted on `/sys/kernel/debugfs`, and if *tracing* is enabled, you should be able to see a directory called *tracing* under *debugfs*. If debugFS is not mounted, you can issue the following command:

```
# mount -t debugfs nodev /sys/kernel/debug
```

If you are unable to see the *tracing* sub-directory, you will need to enable tracing in the kernel configuration, and recompile it. Look for the following options to be enabled in the kernel configuration path (refer to Figure 1):

- i) Kernel Function Tracer (FUNCTION_TRACER)
 - ii) Kernel Function Graph Tracer (FUNCTION_GRAPH_TRACER)
 - iii) Enable/disable Ftrace dynamically (DYNAMIC_FTRACE)
 - iv) Trace max stack (STACK_TRACER)
- Depending on your architecture, a few